

SID4: 1066

PORT_BASE: 8166

PREFIX: s1066

SEED: 1066

VERIFY_SEED: 261066

DOMAIN_ID: 2

Hardware: Macintosh HD, Apple M4, 24 GB, Tahoe 26.6.2

Local Model Used: Ollama qwen3:8b

Part 1)

I: HTML

```
2  <form action="https://webhook.site" method="GET">
3    <h1>Municipal transit incidents</h1>
4    <input type="text" id="incident_id" name="incident_id" required autofocus
5      placeholder="Enter the id of the incident (e.g., INC-00147)">
6    <input type="text" id="route_or_line" name="route_or_line" required
7      placeholder="Enter the affected route or line (e.g., Route 22, Blue Line)">
8    <input type="text" id="email" name="email" required placeholder="Enter your email">
9    <textarea id="description" name="description" required
10      placeholder="Describe what happened (e.g., cause, impact on riders, actions taken)"></textarea>
11    <select name="Incident Type" id="Incident Type">
12      <option value = "Mechanical Breakdown">Mechanical Breakdown</option>
13      <option value = "Collision">Collision</option>
14      <option value = "Delay">Delay</option>
15      <option value = "Route Detour">Route Detour</option>
16    </select>
17
18    <input type="checkbox" id="Acceptance" name="Terms & Conditions" value="yes">
19    <label for="Acceptance">I agree to the terms and conditions.</label>
20
21    <button type="submit">Submit</button>
22    <script src="script.js"></script>
23  </form>
```

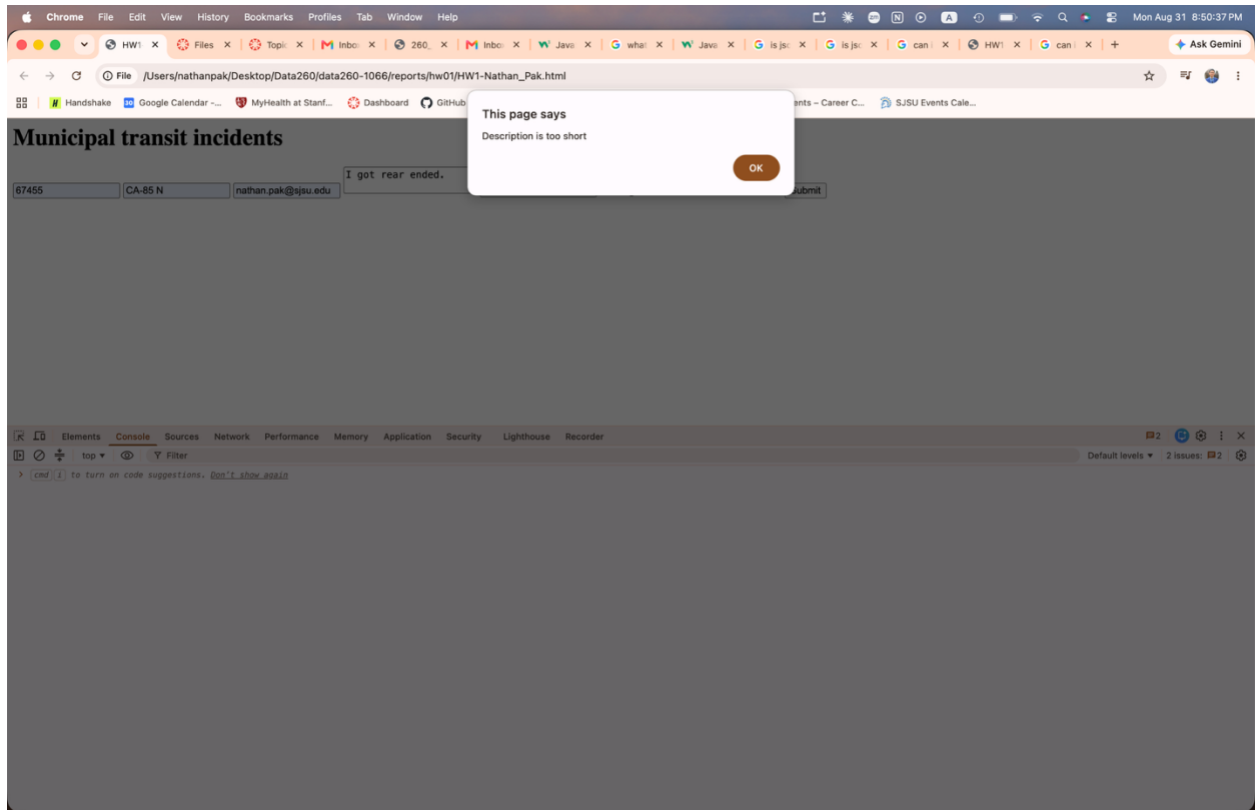
Municipal transit incidents

Enter the id of the incident	Enter the affected route	Enter your email	Describe what happened (e.g., cause, impact)	Mechanical Breakdown	☐ I agree to the terms and conditions.	Submit
---	---	---	---	---	---	---------------------------------------

II: Javascript

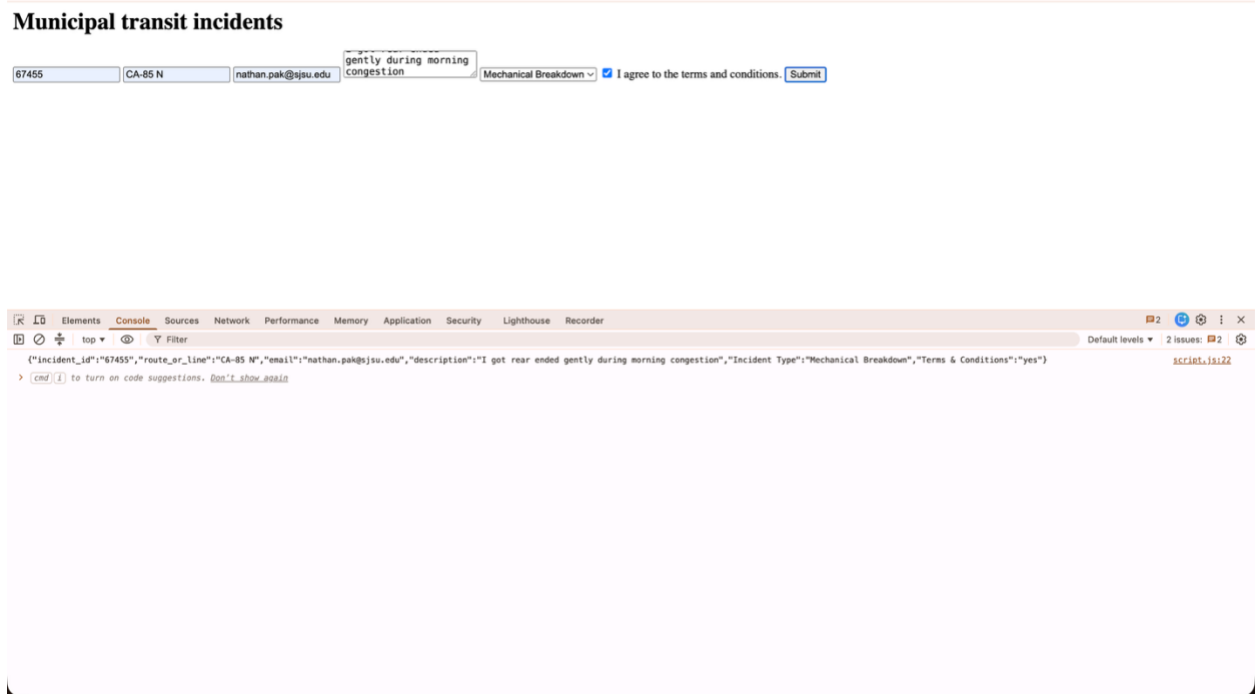
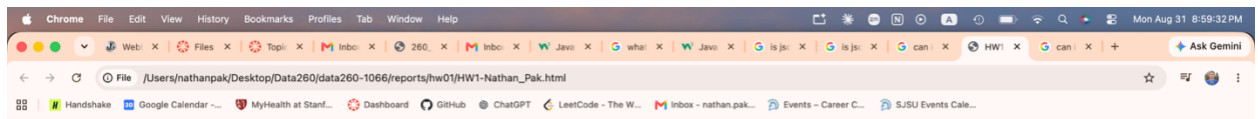
1.

```
s script.js > ...
1  const isValidSubmission = (description, Acceptance) => {
2      if (description.length <= 25) return "Description is too short";
3      const isBoxChecked = document.getElementById(Acceptance).checked;
4      if (isBoxChecked == false) return "Box is not checked"
5      return true;
6  };
7
8  document.querySelector("form").addEventListener("submit", (event) => {
9      const description = document.getElementById("description").value;
10     const result = isValidSubmission(description, "Acceptance");
11
12     if (result !== true) {
13         event.preventDefault();
14         alert(result);
15         return;
16     }
17
18 });
19
20
```



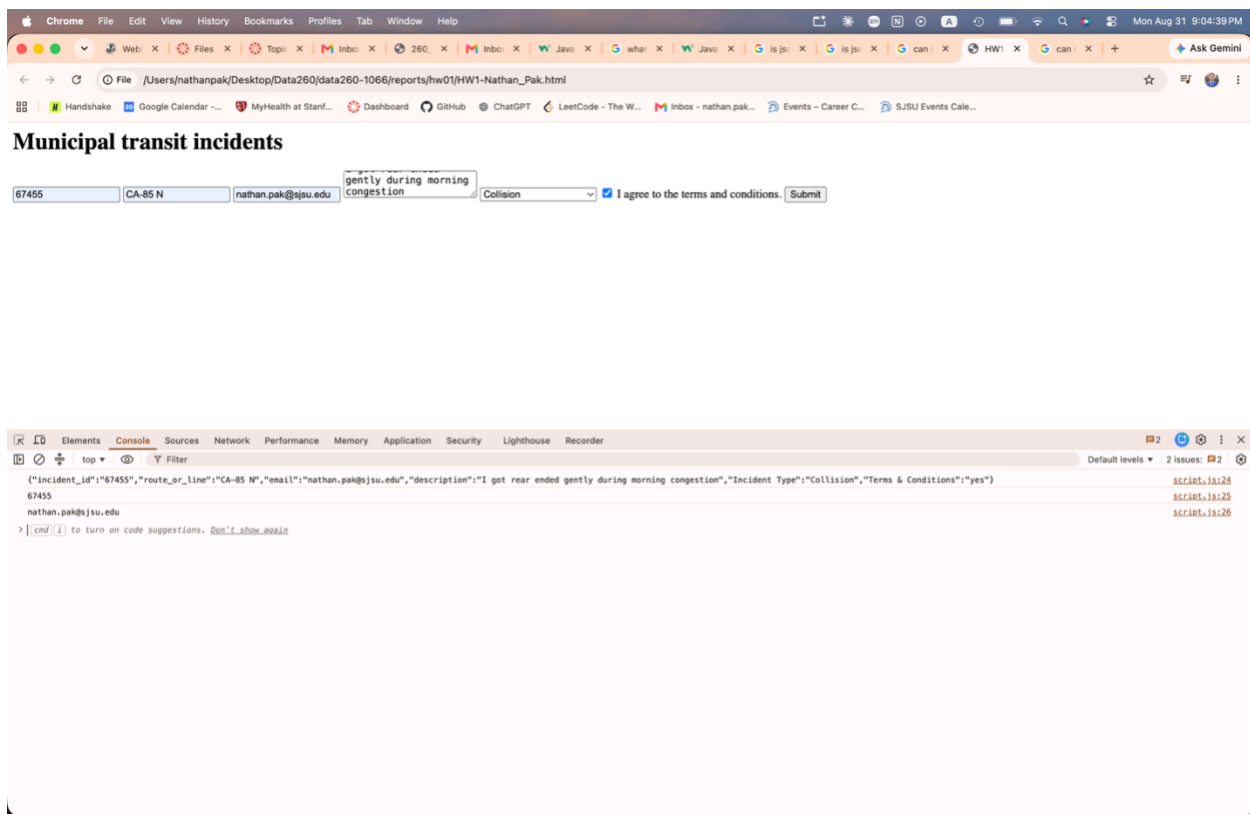
2.

```
1  const isValidSubmission = (description, Acceptance) => {
2    if (description.length <= 25) return "Description is too short";
3    const isChecked = document.getElementById(Acceptance).checked;
4    if (isChecked == false) return "Box is not checked"
5    return true;
6  };
7
8  document.querySelector("form").addEventListener("submit", (event) => {
9    const description = document.getElementById("description").value;
10   const result = isValidSubmission(description, "Acceptance");
11
12   if (result !== true) {
13     event.preventDefault();
14     alert(result);
15     return;
16   }
17
18   const formElement = document.querySelector("form");
19   const formData = new FormData(formElement);
20   const jsonString = JSON.stringify(Object.fromEntries(formData));
21
22   console.log(jsonString);
23
24   event.preventDefault();
25 });
26
27
```

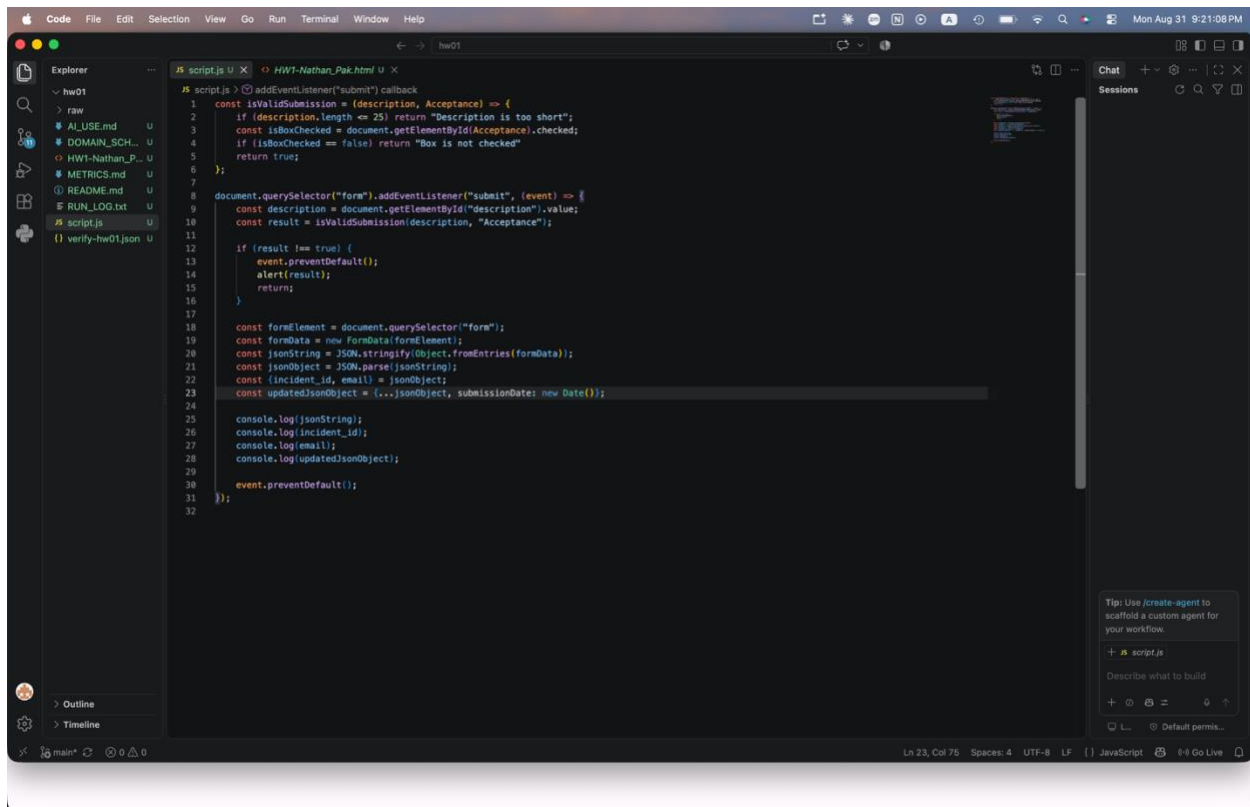


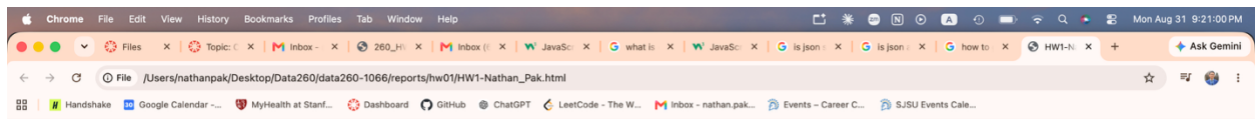
3.

```
1  ✓ const isValidSubmission = (description, Acceptance) => {
2    if (description.length <= 25) return "Description is too short";
3    const isBoxChecked = document.getElementById(Acceptance).checked;
4    if (isBoxChecked == false) return "Box is not checked"
5    return true;
6  };
7
8  ✓ document.querySelector("form").addEventListener("submit", (event) => {
9    const description = document.getElementById("description").value;
10   const result = isValidSubmission(description, "Acceptance");
11
12   ✓ if (result !== true) {
13     event.preventDefault();
14     alert(result);
15     return;
16   }
17
18   const formElement = document.querySelector("form");
19   const formData = new FormData(formElement);
20   const jsonString = JSON.stringify(Object.fromEntries(formData));
21   const jsonObject = JSON.parse(jsonString);
22   const {incident_id, email} = jsonObject;
23
24   console.log(jsonString);
25   console.log(incident_id);
26   console.log(email);
27
28   event.preventDefault();
29   });
30
```



4.



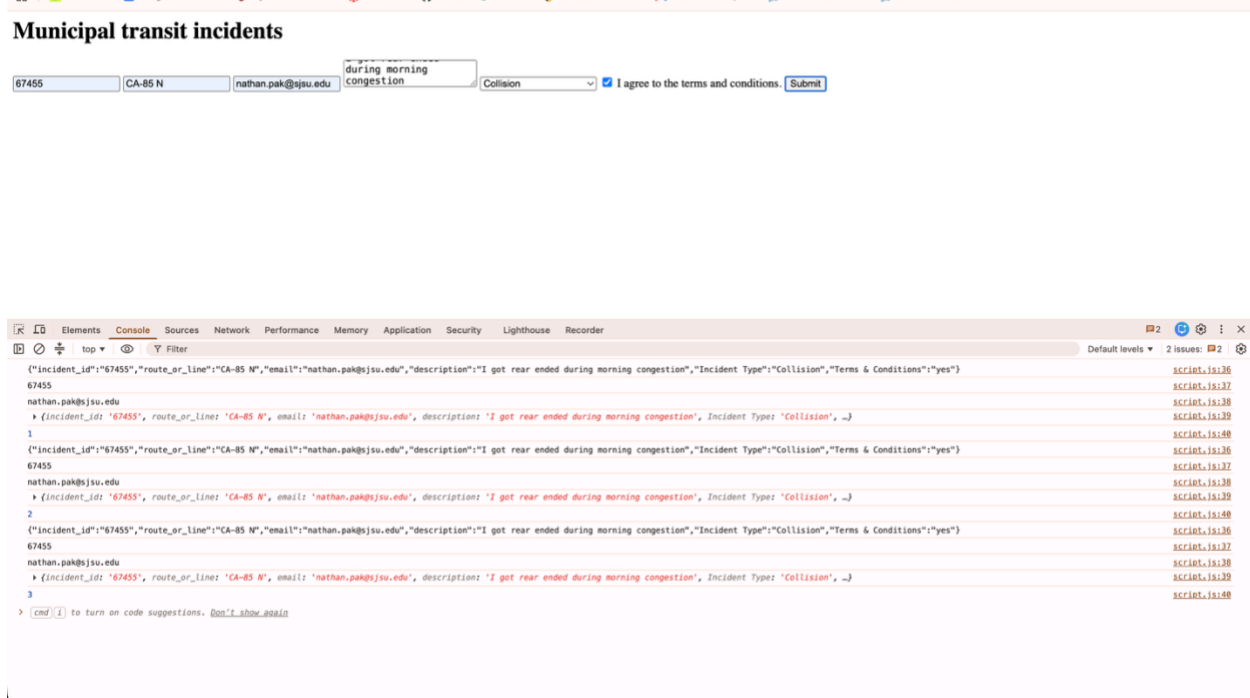
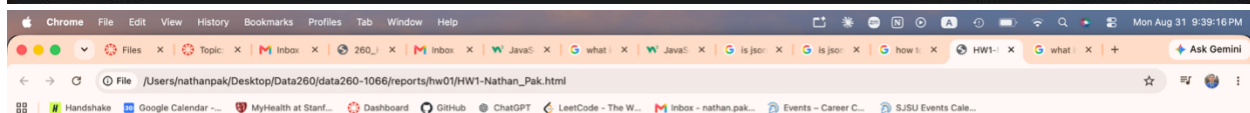
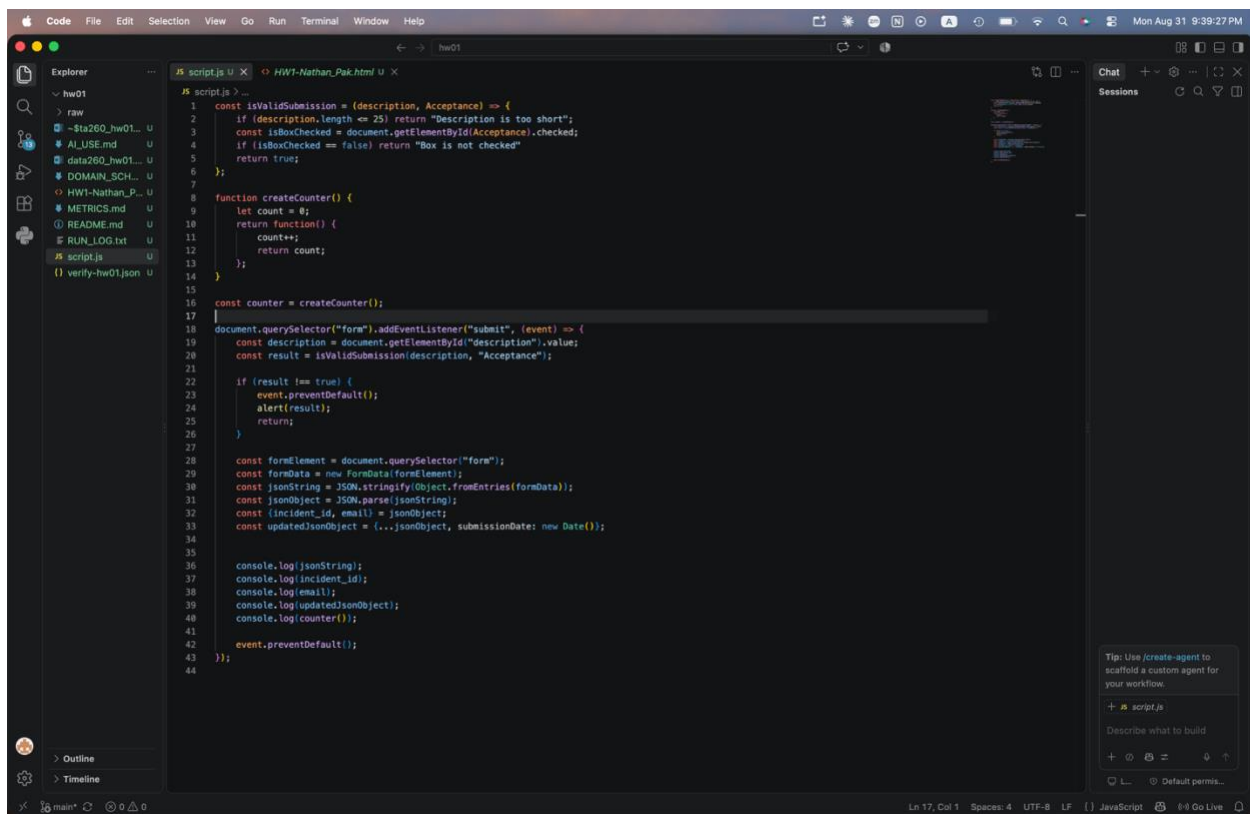


Municipal transit incidents

67455 CA-85 N nathan.pak@sjsu.edu gently during morning congestion Collision ☒ I agree to the terms and conditions. Submit



5.



Screenshot of app running on localhost via Docker:

The screenshot shows a web browser window with the address bar set to `localhost:8000`. The page title is "Municipal transit incidents". Below the title is a form with the following fields and elements:

- Enter the id of the incident
- Enter the affected route
- Enter your email
- Describe what happened (e.g., cause, impact)
- Mechanical Breakdown
- ☐ I agree to the terms and conditions
-

Screenshot of app running on public IP Address via AWS ECS:

Chrome File Edit View History Bookmarks Profiles Tab Window Help Tue Sep 1 12:22:52 AM

Not Secure 18.220.65.140:8000 Ask Gemini

Handshake Google Calendar ~... MyHealth at Stanf... Dashboard GitHub ChatGPT LeetCode - The W... Inbox - nathan.pak... Events - Career C... SJSU Events Cale...

Municipal transit incidents

☐ I agree to the terms and conditions.

Part 2)

1 & 2)

```

1  import json
2  import sys
3  import requests
4
5  OLLAMA_URL = "http://localhost:11434/api/chat"
6  MODEL = "qwen3:8b"
7  MAX_SUMMARY_WORDS = 25
8  NUM_TAGS = 3
9  MAX_RETRIES = 3
10
11
12  def call_model(system_prompt: str, user_prompt: str) -> dict:
13
14      payload = {
15          "model": MODEL,
16          "messages": [
17              {"role": "system", "content": system_prompt},
18              {"role": "user", "content": user_prompt},
19          ],
20          "format": "json",
21          "stream": False,
22          "think": False,
23      }
24
25      last_error = None
26      for attempt in range(1, MAX_RETRIES + 1):
27          try:
28              resp = requests.post(OLLAMA_URL, json=payload, timeout=240)
29              resp.raise_for_status()
30              content = resp.json()["message"]["content"]
31              return json.loads(content)
32          except (requests.RequestException, KeyError, json.JSONDecodeError) as exc:
33              last_error = exc
34              print(f" [retry {attempt}/{MAX_RETRIES}] model call failed: {exc}",
35                    file=sys.stderr)
36
37      raise RuntimeError(f"Model call failed after {MAX_RETRIES} attempts: {last_error}")
38
39

```

```

39
40 def planner_step(title: str, content: str) -> dict:
41
42     system_prompt = (
43         "You are the Planner agent in a small multi-agent pipeline. "
44         "Given a title and content describing some record or event, "
45         "propose 5 to 7 candidate topical tags (short lowercase phrases) "
46         "that are directly grounded in specific details from the given "
47         "content, and a rough draft summary (1-3 sentences, will be "
48         "trimmed later). Base every tag and the summary strictly on what "
49         "the title/content actually say – do not invent topics that "
50         "aren't supported by the text. "
51         "Respond ONLY with JSON in this exact shape: "
52         '{"candidate_tags": ["...", "...", ...], "draft_summary": "..."}'
53     )
54     user_prompt = f"Title: {title}\nContent: {content}"
55     return call_model(system_prompt, user_prompt)
56
57
58 def reviewer_step(planner_output: dict, title: str, content: str) -> dict:
59
60     system_prompt = (
61         "You are the Reviewer agent in a small multi-agent pipeline. You "
62         "will receive a title, content, and a Planner's draft (candidate "
63         f"tags + draft summary). Select exactly {NUM_TAGS} of the most "
64         "topical tags from the candidates, keeping only ones clearly "
65         "grounded in the content (you may lightly reword them, but "
66         f"prefer the given candidates). Rewrite the summary to be at "
67         f"most {MAX_SUMMARY_WORDS} words, factual and specific to the "
68         "content – do not add information that isn't in the title/content. "
69         "Respond ONLY with JSON in this exact shape: "
70         '{"tags": ["...", "...", "..."], "summary": "..."}'
71     )
72     user_prompt = (
73         f"Title: {title}\nContent: {content}\n\n"
74         f"Planner draft: {json.dumps(planner_output)}"
75     )
76     return call_model(system_prompt, user_prompt)
77
78

```

```

77
78
79 def finalizer_step(reviewer_output: dict, title: str, content: str) -> dict:
80
81     tags = reviewer_output.get("tags", [])
82     summary = reviewer_output.get("summary", "")
83
84     if _is_valid(tags, summary):
85         return {"tags": tags, "summary": summary}
86
87     try:
88         system_prompt = (
89             "You are the Finalizer agent. The previous output violated the "
90             f"required contract: exactly {NUM_TAGS} tags and a summary of at "
91             f"most {MAX_SUMMARY_WORDS} words. Fix it using only the given "
92             "title/content as grounding – do not introduce new topics. "
93             "Respond ONLY with JSON in this exact shape: "
94             '{"tags": [...], "summary": "..."}'
95         )
96         user_prompt = (
97             f"Title: {title}\nContent: {content}\n\n"
98             f"Invalid output to fix: {json.dumps(reviewer_output)}"
99         )
100         fixed = call_model(system_prompt, user_prompt)
101         tags = fixed.get("tags", tags)
102         summary = fixed.get("summary", summary)
103     except RuntimeError as exc:
104         print(f" Finalizer corrective call failed, using local fallback: {exc}",
105               file=sys.stderr)
106
107     if _is_valid(tags, summary):
108         return {"tags": tags, "summary": summary}
109
110     tags = (tags + ["general"] * NUM_TAGS)[:NUM_TAGS]
111     words = summary.split()
112     if len(words) > MAX_SUMMARY_WORDS:
113         summary = " ".join(words[:MAX_SUMMARY_WORDS])
114     return {"tags": tags, "summary": summary}
115
116

```

```

116
117 def _is_valid(tags, summary) -> bool:
118     return (
119         isinstance(tags, list)
120         and len(tags) == NUM_TAGS
121         and all(isinstance(t, str) and t.strip() for t in tags)
122         and isinstance(summary, str)
123         and 0 < len(summary.split()) <= MAX_SUMMARY_WORDS
124     )
125
126
127 def run_pipeline(title: str, content: str) -> dict:
128     print("Planner working...", file=sys.stderr)
129     planner_output = planner_step(title, content)
130     print(f" Planner output: {json.dumps(planner_output)}", file=sys.stderr)
131
132     print("Reviewer working...", file=sys.stderr)
133     reviewer_output = reviewer_step(planner_output, title, content)
134     print(f" Reviewer output: {json.dumps(reviewer_output)}", file=sys.stderr)
135
136     print("Finalizer working...", file=sys.stderr)
137     final_output = finalizer_step(reviewer_output, title, content)
138     return final_output
139
140
141 if __name__ == "__main__":
142     # Example input matching the Municipal Transit Incident domain schema
143     # (route_or_line as the primary field, description as the content field).
144     sample_title = "Route 22 Bus Breakdown Near Downtown Transit Center"
145     sample_content = (
146         "A Route 22 bus experienced a mechanical breakdown near the "
147         "Downtown Transit Center during evening rush hour. The engine "
148         "stalled at a signal and could not restart, blocking one lane of "
149         "traffic. Riders were transferred to the following bus after a "
150         "20-minute delay. A maintenance crew towed the disabled vehicle "
151         "off-route roughly 45 minutes after the initial report."
152     )
153
154     result = run_pipeline(sample_title, sample_content)
155     print(json.dumps(result))
156

```

```
(base) Nathans-MacBook-Pro:hw01 nathanpak$ python agents_demo.py
Planner working...
Planner output: {"candidate_tags": ["bus breakdown", "downtown transit center", "mechanical failure", "evening rush hour", "traffic lane blocked", "maintenance crew response", "riders transferred"], "draft_summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, causing a traffic lane blockage. Riders were transferred to a following bus after a 20-minute delay, with the disabled vehicle towed off-route 45 minutes later."}
Reviewer working...
Reviewer output: {"tags": ["bus breakdown", "downtown transit center", "traffic lane blocked"], "summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking a traffic lane."}
Finalizer working...
{"tags": ["bus breakdown", "downtown transit center", "traffic lane blocked"], "summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking a traffic lane."}
(base) Nathans-MacBook-Pro:hw01 nathanpak$
```

3)

Q1. The 3 final tags you obtained.

bus breakdown, downtown transit center, traffic lane blocked

Q2. The final summary (≤25 words).

A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking a traffic lane.

Q3. Did the Reviewer agent change anything? (yes/no + explanation)

Yes. The reviewer agent narrows the tag count down to three most relevant tags and tightens the summary to no more than 25 words.

4)

The planner reads the title and content input and generates a generous amount of tags along with a draft summary of the input data relevant to the assigned domain entity.

The reviewer step takes in the output of the planner agent and chooses the three most relevant tags out of the list that fit with the given domain of the input. The reviewer also shortens the summary to at most 25 words.

The finalizer agent validates the output from the reviewer step. Unlike the previous two steps, it doesn't call the LLM model. It makes sure that the summary is at most 25 words and there are just 3 tags. If it is invalid, it first asks the model by giving instructions to correct

the input. If it still fails, it deterministically truncates the input so that the input data becomes valid. Then, it outputs the new transformed data.

Part 3)

1&2)

```
1  import json
2  import os
3  import statistics
4  import time
5  from collections import Counter
6
7  # Import the pipeline components directly from your existing script
8  import agents_demo
9
10 SAMPLE_TITLE = "Route 22 Bus Breakdown Near Downtown Transit Center"
11 SAMPLE_CONTENT = (
12     "A Route 22 bus experienced a mechanical breakdown near the "
13     "Downtown Transit Center during evening rush hour. The engine "
14     "stalled at a signal and could not restart, blocking one lane of "
15     "traffic. Riders were transferred to the following bus after a "
16     "20-minute delay. A maintenance crew towed the disabled vehicle "
17     "off-route roughly 45 minutes after the initial report."
18 )
19
20 OUTPUT_DIR = os.path.join("reports", "hw01", "raw")
21 OUTPUT_FILE = os.path.join(OUTPUT_DIR, "per_run_results.json")
22
23
24 def calculate_percentiles(values):
25     """Calculates p50, p95, and p99 from a sorted list of values."""
26     sorted_v = sorted(values)
27     n = len(sorted_v)
28     if n == 0:
29         return {"p50": 0.0, "p95": 0.0, "p99": 0.0}
30
31     def get_percentile(p):
32         k = (n - 1) * (p / 100.0)
33         f = int(k)
34         c = f + 1
35         if c < n:
36             return sorted_v[f] + (k - f) * (sorted_v[c] - sorted_v[f])
37         return sorted_v[f]
38
39     return {
40         "p50": round(get_percentile(50), 4),
41         "p95": round(get_percentile(95), 4),
42         "p99": round(get_percentile(99), 4),
43     }
44
```

```
45
46 def analyze_temperature_runs(runs_data):
47     # 1. Distinct Tag Sets (using frozenset to ignore order differences)
48     tag_sets = [frozenset(r["tags"]) for r in runs_data]
49     distinct_tag_sets = len(set(tag_sets))
50
51     # 2. Tag frequencies across runs
52     all_tags = [tag for r in runs_data for tag in r["tags"]]
53     tag_counts = Counter(all_tags)
54
55     tags_in_all = [tag for tag, count in tag_counts.items() if count == 20]
56     tags_in_one = [tag for tag, count in tag_counts.items() if count == 1]
57
58     # 3. Latency Metrics
59     latencies = [r["latency_seconds"] for r in runs_data]
60     percentiles = calculate_percentiles(latencies)
61
62     return {
63         "distinct_tag_sets_count": distinct_tag_sets,
64         "tags_in_all_20_runs": tags_in_all,
65         "tags_in_exactly_one_run": tags_in_one,
66         "latency_percentiles": percentiles,
67     }
68
```

```

69
70 def main():
71     temperatures = [0.7, 0.0]
72     runs_per_temp = 20
73     all_raw_data = []
74
75     print("Starting evaluation runs...")
76
77     for temp in temperatures:
78         print(f"\n--- Starting 20 runs at Temperature {temp} ---")
79         for run_idx in range(1, runs_per_temp + 1):
80             print(f"Run {run_idx}/20 (temp={temp})...")
81             start_time = time.perf_counter()
82
83             # Execute the multi-agent pipeline step
84             result = agents_demo.run_pipeline(
85                 SAMPLE_TITLE, SAMPLE_CONTENT, temperature=temp
86             )
87
88             latency = time.perf_counter() - start_time
89
90             all_raw_data.append(
91                 {
92                     "run_id": len(all_raw_data) + 1,
93                     "temperature": temp,
94                     "run_index": run_idx,
95                     "tags": result.get("tags", []),
96                     "summary": result.get("summary", ""),
97                     "latency_seconds": round(latency, 4),
98                 }
99             )
100
101     # Ensure output directory exists and save raw data
102     os.makedirs(OUTPUT_DIR, exist_ok=True)
103     with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
104         json.dump(all_raw_data, f, indent=2)
105
106     print(f"\nRaw results successfully saved to: {OUTPUT_FILE}\n")
107
108     # Generate and print reports
109     print("=" * 60)
110     print("EVALUATION METRICS REPORT")
111     print("=" * 60)
112

```

```

112
113     for temp in temperatures:
114         temp_runs = [r for r in all_raw_data if r["temperature"] == temp]
115         metrics = analyze_temperature_runs(temp_runs)
116
117         print(f"\nResults for Temperature {temp}:")
118         print(
119             f"    • Number of Distinct Tag Sets: {metrics['distinct_tag_sets_count']}"
120         )
121         print(f"    • Tags in ALL 20 runs: {metrics['tags_in_all_20_runs']}")
122         print(
123             f"    • Tags in EXACTLY 1 run: {metrics['tags_in_exactly_one_run']}"
124         )
125         print(
126             f"    • Latency p50: {metrics['latency_percentiles']['p50']} seconds"
127         )
128         print(
129             f"    • Latency p95: {metrics['latency_percentiles']['p95']} seconds"
130         )
131         print(
132             f"    • Latency p99: {metrics['latency_percentiles']['p99']} seconds"
133         )
134
135         print("\n" + "=" * 60)
136
137
138 if __name__ == "__main__":
139     main()

```

Raw results successfully saved to: reports/hw01/raw/per_run_results.json

```

=====
EVALUATION METRICS REPORT
=====

```

Results for Temperature 0.7:

- Number of Distinct Tag Sets: 2
- Tags in ALL 20 runs: ['bus breakdown', 'downtown transit center']
- Tags in EXACTLY 1 run: []
- Latency p50: 15.0353 seconds
- Latency p95: 16.3034 seconds
- Latency p99: 24.3008 seconds

Results for Temperature 0.0:

- Number of Distinct Tag Sets: 1
- Tags in ALL 20 runs: ['bus breakdown', 'downtown transit center', 'traffic blockage']
- Tags in EXACTLY 1 run: []
- Latency p50: 14.2395 seconds
- Latency p95: 15.0961 seconds
- Latency p99: 15.2013 seconds

3)

```

raw > {} per_run_results.json > ...
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
{
  "run_id": 1,
  "temperature": 0.7,
  "run_index": 1,
  "tags": [
    "traffic lane blocked"
  ],
  "summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking a traffic lane.",
  "latency_seconds": 26.3801
},
{
  "run_id": 2,
  "temperature": 0.7,
  "run_index": 2,
  "tags": [
    "bus breakdown",
    "downtown transit center",
    "traffic lane blocked"
  ],
  "summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking a traffic lane.",
  "latency_seconds": 15.1257
},
{
  "run_id": 3,
  "temperature": 0.7,
  "run_index": 3,
  "tags": [
    "bus breakdown",
    "downtown transit center",
    "traffic blockage"
  ],
  "summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking traffic and delaying riders.",
  "latency_seconds": 14.9886
},
{
  "run_id": 4,
  "temperature": 0.7,
  "run_index": 4,
  "tags": [
    "bus breakdown",
    "downtown transit center",
    "traffic lane blocked"
  ],
  "summary": "A Route 22 bus broke down near the Downtown Transit Center during evening rush hour, blocking a traffic lane.",
  "latency_seconds": 15.1257
}

```

4) Using your own numbers, explain what two users sending identical input might see, and give one example where run-to-run variation is acceptable and one where it is not.

At temperature 0.0, two users sending identical input will receive 100% identical tag outputs. Meanwhile for temperature 0.7, two users sending identical input may receive slightly different outputs. While both users are guaranteed to see 'bus breakdown' and 'downtown transit center', the third tag will vary. That is why there are two distinct tag sets across 20 runs. Run to run variation is acceptable in creative content generation. When we want many different ideas for coming up ways to solve a problem, variations across runs can be helpful. However, for compliance auditing, we want identical inputs to map strictly to identical outputs in order to ensure correct information for legal or financial services.

Part 4)

1&2) Create hw1_client.py as a small command-line demo that imports the adapter.

```

1 import argparse
2 import json
3 import sys
4 from src.model_client import complete
5
6
7 def main():
8     parser = argparse.ArgumentParser(
9         description="CLI demo testing the src.model_client adapter."
10    )
11    parser.add_argument(
12        "--prompt",
13        type=str,
14        default="Summarize the importance of clear software interfaces in 2 sentences.",
15        help="User prompt to send to the model.",
16    )
17    parser.add_argument(
18        "--temperature",
19        type=float,
20        default=0.0,
21        help="Sampling temperature (default: 0.0).",
22    )
23    parser.add_argument(
24        "--test-tools",
25        action="store_true",
26        help="Demonstrate the tools parameter interface.",
27    )
28    args = parser.parse_args()
29
30    print("---- Executing Model Client Adapter Demo ----")
31
32    if args.test_tools:
33        # Example tool definition to verify tool schema passing
34        sample_tools = [
35            {
36                "type": "function",
37                "function": {
38                    "name": "lookup_transit_status",
39                    "description": "Get current delay status for a transit line",
40                    "parameters": {
41                        "type": "object",
42                        "properties": {
43                            "line_name": {
44                                "type": "string",
45                                "description": "The name or number of the bus/train line",
46                            }
47                        }
48                    }
49                }
50            }
51        ]

```

```

45         "description": "The name or number of the bus/train line",
46     },
47 },
48     "required": ["line_name"],
49 },
50 },
51 }
52 ]
53
54 messages = [
55     {
56         "role": "user",
57         "content": "Can you check the current status of Route 22?",
58     }
59 ]
60
61 print(
62     f"Sending payload with tools defined (Temperature: {args.temperature})...\n"
63 )
64 response = complete(
65     messages=messages,
66     tools=sample_tools,
67     temperature=args.temperature,
68     response_format=None,
69 )
70
71 else:
72     messages = [
73         {
74             "role": "system",
75             "content": "You are a concise AI assistant. Respond ONLY with JSON containing a single key 'response'.",
76         },
77         {"role": "user", "content": args.prompt},
78     ]
79
80 print(
81     f"Sending structured prompt (Temperature: {args.temperature})...\n"
82 )
83 response = complete(
84     messages=messages,
85     temperature=args.temperature,
86     response_format="json",
87 )
88
89 print("Adapter Output:")
90 print(json.dumps(response, indent=2))

```

```

88     print("Adapter Output:")
89     print(json.dumps(response, indent=2))
90
91
92 ✓ if __name__ == "__main__":
93     main()

```

```
[(base) Nathans-MacBook-Pro:hw01 nathanpak$ python hw1_client.py --prompt "Name three core trans-
sit metrics." --temperature 0.0
--- Executing Model Client Adapter Demo ---
Sending structured prompt (Temperature: 0.0)...

Adapter Output:
{
  "response": [
    "On-time performance",
    "Passenger load factor",
    "Frequency of service"
  ]
}
(base) Nathans-MacBook-Pro:hw01 nathanpak$ █
```

3. Create AGENT.md asking for strict bullet-only code review, and verify the model actually follows it.

verify_agent.py > ...

```
1  import sys
2  import os
3  from src.model_client import complete
4
5  # Sample code with intentional issues to review
6  CODE_TO_REVIEW = """
7  def process_user_data(data):
8      # Bug: SQL injection risk
9      query = "SELECT * FROM users WHERE id = '" + data['id'] + "'"
10
11     # Bug: Bare except
12     try:
13         result = execute_query(query)
14         return result
15     except:
16         return None
17 """
18
19 def verify_compliance(response_text: str) -> tuple[bool, list[str]]:
20     """Checks whether the response strictly adheres to AGENT.md formatting rules."""
21     lines = [line.strip() for line in response_text.strip().split("\n") if line.strip()]
22     violations = []
23
24     for idx, line in enumerate(lines, 1):
25         # 1. Must start with a bullet character
26         if not (line.startswith("* ") or line.startswith("- ")):
27             violations.append(f"Line {idx} does not start with a bullet point: '{line[:40]}...'")
28
29         # 2. Must not contain Markdown headers
30         if line.startswith("#"):
31             violations.append(f"Line {idx} uses Markdown header formatting: '{line[:20]}...'")
32
33     is_compliant = len(violations) == 0
34     return is_compliant, violations
35
36
37 def main():
38     agent_md_path = "AGENT.md"
39     if not os.path.exists(agent_md_path):
40         print(f"Error: {agent_md_path} not found. Please create it first.")
41         sys.exit(1)
42
43     with open(agent_md_path, "r", encoding="utf-8") as f:
44         agent_instructions = f.read()
45
```

```

45
46     messages = [
47         {"role": "system", "content": agent_instructions},
48         {
49             "role": "user",
50             "content": f>Please review this code:\n\n``python\n{CODE_TO_REVIEW}\n``",
51         },
52     ]
53
54     print("Sending code review request using AGENT.md system prompt...\n")
55
56     # Request standard (parameter) messages: List[Dict[str, str]]
57     result = complete(messages=messages, temperature=0.0, response_format=None)
58     review_output = result.get("content", "")
59
60     print("--- Model Output Start ---")
61     print(review_output)
62     print("--- Model Output End ---\n")
63
64     # Verify formatting constraints
65     is_compliant, violations = verify_compliance(review_output)
66
67     print("=" * 50)
68     print("VERIFICATION RESULT")
69     print("=" * 50)
70     if is_compliant:
71         print(" SUCCESS: Model strictly followed the AGENT.md bullet-only protocol!")
72     else:
73         print(" FAILED: Model violated formatting rules:")
74         for v in violations:
75             print(f"   - {v}")
76     print("=" * 50)
77
78
79 if __name__ == "__main__":
80     main()

```

```

[(base) Nathans-MacBook-Pro:hw01 nathanpak$ python verify_agent.py
Sending code review request using AGENT.md system prompt...

--- Model Output Start ---
* **Security:** The SQL query in line 4 is vulnerable to SQL injection attacks because it directly concatenates user input without parameterization.
* **Security:** The `execute_query` function is not shown, but the use of raw SQL with user input is a significant security risk.
* **Error Handling:** The `except` block on line 8 catches all exceptions, which is a bare except clause and can hide unexpected errors. It should be specific to relevant exceptions.
* **Correctness:** The function does not handle cases where `data` might not have an `id` key, leading to a `KeyError`.
--- Model Output End ---

=====
VERIFICATION RESULT
=====
SUCCESS: Model strictly followed the AGENT.md bullet-only protocol!
=====
(base) Nathans-MacBook-Pro:hw01 nathanpak$ █

```

4) After every model response, print input tokens, output tokens, and total tokens for that turn. On exit, print cumulative input tokens, output tokens, and turn count.

```

hw1_client.py > ...
1  ∨ import atexit
2    import sys
3    from src.model_client import complete
4
5    # Cumulative counter state
6    cumulative_input_tokens = 0
7    cumulative_output_tokens = 0
8    turn_count = 0
9
10
11  ∨ def print_summary_on_exit():
12      """Triggered on script exit to display total conversation usage stats."""
13      ∨ if turn_count > 0:
14          total_cumulative = cumulative_input_tokens + cumulative_output_tokens
15          print("\n" + "=" * 50)
16          print("CUMULATIVE SESSION STATS")
17          print("=" * 50)
18          print(f"Total Turns: {turn_count}")
19          print(f"Cumulative Input Tokens: {cumulative_input_tokens}")
20          print(f"Cumulative Output Tokens: {cumulative_output_tokens}")
21          print(f"Cumulative Total Tokens: {total_cumulative}")
22          print("=" * 50)
23
24
25      # Register exit hook
26      atexit.register(print_summary_on_exit)
27
28
29  ∨ def main():
30      global cumulative_input_tokens, cumulative_output_tokens, turn_count
31
32      print("Interactive Chat Loop (Type 'exit' or 'quit' to exit)\n")
33
34      ∨ while True:
35          ∨ try:
36              user_input = input("User > ").strip()
37              ∨ if not user_input:
38                  continue
39              ∨ if user_input.lower() in ("exit", "quit"):
40                  break
41
42              turn_count += 1
43              ∨ messages = [
44                  {"role": "system", "content": "You are a helpful assistant."},
45                  {"role": "user", "content": user_input},
46              ]
47

```

```

47
48     # Request text completion
49     response = complete(messages=messages, response_format=None)
50
51     content = response.get("content", "")
52     usage = response.get("_usage", {})
53
54     in_tok = usage.get("input_tokens", 0)
55     out_tok = usage.get("output_tokens", 0)
56     tot_tok = usage.get("total_tokens", 0)
57
58     # Update cumulative counters
59     cumulative_input_tokens += in_tok
60     cumulative_output_tokens += out_tok
61
62     print(f"\nAssistant > {content}\n")
63     print(
64         f"[Turn {turn_count} Usage] Input Tokens: {in_tok} | "
65         f"Output Tokens: {out_tok} | Total Tokens: {tot_tok}\n"
66     )
67
68     except (KeyboardInterrupt, EOFError):
69         break
70
71
72 if __name__ == "__main__":
73     main()

```

```

}
[(base) Nathans-MacBook-Pro:hw01 nathanpak$ python hw1_client.py
Interactive Chat Loop (Type 'exit' or 'quit' to exit)

User > Hello, explain what an API is in one sentence.

Assistant > An API (Application Programming Interface) is a set of rules and protocols that al
lows different software applications to communicate and interact with each other.

[Turn 1 Usage] Input Tokens: 38 | Output Tokens: 28 | Total Tokens: 66

User > Give me two examples of APIs.

Assistant > Sure! Here are two examples of APIs:

1. **Google Maps API**
   This API allows developers to integrate Google Maps into their applications, enabling featu
res like displaying maps, getting directions, and finding locations.

2. **Twitter API (X API)**
   The Twitter API allows developers to access Twitter data, such as posting tweets, retrievin
g tweets, and managing user accounts, enabling integration with other applications or services
.

[Turn 2 Usage] Input Tokens: 34 | Output Tokens: 86 | Total Tokens: 120

User > exit

=====
CUMULATIVE SESSION STATS
=====
Total Turns: 2
Cumulative Input Tokens: 72
Cumulative Output Tokens: 114
Cumulative Total Tokens: 186
=====
(base) Nathans-MacBook-Pro:hw01 nathanpak$ █

```

5 & 6) Add a /stats command showing turn count, cumulative token counts, and serialized conversation-history length, without altering the history. Run a five-turn conversation; record

/stats output after turn 3 and after turn 5.

```
hw1_client.py > ...
1  import atexit
2  import json
3  import sys
4  from src.model_client import complete
5
6  # Conversation and metric state
7  messages = [
8      {"role": "system", "content": "You are a helpful assistant."}
9  ]
10 cumulative_input_tokens = 0
11 cumulative_output_tokens = 0
12 turn_count = 0
13
14
15 def print_summary_on_exit():
16     """Triggered on script exit to display total conversation usage stats."""
17     if turn_count > 0:
18         total_cumulative = cumulative_input_tokens + cumulative_output_tokens
19         print("\n" + "=" * 50)
20         print("CUMULATIVE SESSION STATS")
21         print("=" * 50)
22         print(f"Total Turns: {turn_count}")
23         print(f"Cumulative Input Tokens: {cumulative_input_tokens}")
24         print(f"Cumulative Output Tokens: {cumulative_output_tokens}")
25         print(f"Cumulative Total Tokens: {total_cumulative}")
26         print("=" * 50)
27
28
29 # Register exit hook
30 atexit.register(print_summary_on_exit)
31
32
33 def print_current_stats():
34     """Prints live stats including serialized conversation history length."""
35     serialized_history = json.dumps(messages)
36     history_char_length = len(serialized_history)
37     total_tokens = cumulative_input_tokens + cumulative_output_tokens
38
39     print("\n" + "-" * 40)
40     print("LIVE STATS REPORT")
41     print("-" * 40)
42     print(f"Turn Count: {turn_count}")
43     print(f"Cumulative Input Tokens: {cumulative_input_tokens}")
44     print(f"Cumulative Output Tokens: {cumulative_output_tokens}")
45     print(f"Cumulative Total Tokens: {total_tokens}")
```

```

44     print(f"Cumulative Output Tokens: {cumulative_output_tokens}")
45     print(f"Cumulative Total Tokens: {total_tokens}")
46     print(f"Serialized History Length: {history_char_length} characters")
47     print(f"Message Stack Count: {len(messages)} messages")
48     print("-" * 40 + "\n")
49
50
51 def main():
52     global cumulative_input_tokens, cumulative_output_tokens, turn_count, messages
53
54     print("Interactive Chat Loop (Commands: '/stats', 'exit', 'quit')\n")
55
56     while True:
57         try:
58             user_input = input("User > ").strip()
59             if not user_input:
60                 continue
61
62             # Command: Exit
63             if user_input.lower() in ("exit", "quit"):
64                 break
65
66             # Command: /stats
67             if user_input.lower() == "/stats":
68                 print_current_stats()
69                 continue
70
71             # Append user turn to ongoing conversation history
72             messages.append({"role": "user", "content": user_input})
73             turn_count += 1
74
75             # Request text completion passing full conversation history
76             response = complete(messages=messages, response_format=None)
77
78             content = response.get("content", "")
79             usage = response.get("_usage", {})
80
81             # Append assistant response to history
82             messages.append({"role": "assistant", "content": content})
83
84             in_tok = usage.get("input_tokens", 0)
85             out_tok = usage.get("output_tokens", 0)
86             tot_tok = usage.get("total_tokens", 0)
87

```

```

87
88         # Accumulate token metrics
89         cumulative_input_tokens += in_tok
90         cumulative_output_tokens += out_tok
91
92         print(f"\nAssistant > {content}\n")
93         print(
94             f"[Turn {turn_count} Usage] Input Tokens: {in_tok} | "
95             f"Output Tokens: {out_tok} | Total Tokens: {tot_tok}\n"
96         )
97
98     except (KeyboardInterrupt, EOFError):
99         break
100
101
102 if __name__ == "__main__":
103     main()

```

```

[(base) Nathans-MacBook-Pro:hw01 nathanpak$ python hw1_client.py
Interactive Chat Loop (Commands: '/stats', 'exit', 'quit')

```

User > Hello!

Assistant > Hello! How can I assist you today? 😊

[Turn 1 Usage] Input Tokens: 29 | Output Tokens: 12 | Total Tokens: 41

User > /stats

LIVE STATS REPORT

Turn Count:	1
Cumulative Input Tokens:	29
Cumulative Output Tokens:	12
Cumulative Total Tokens:	41
Serialized History Length:	187 characters
Message Stack Count:	3 messages

[Turn 2 Usage] Input Tokens: 54 | Output Tokens: 233 | Total Tokens: 287

User > No thanks

Assistant > You're welcome! If you have any other questions, feel free to ask.



[Turn 3 Usage] Input Tokens: 298 | Output Tokens: 19 | Total Tokens: 317

User > /stats

LIVE STATS REPORT

Turn Count:	3
Cumulative Input Tokens:	380
Cumulative Output Tokens:	264
Cumulative Total Tokens:	644
Serialized History Length:	1600 characters
Message Stack Count:	7 messages

User > █

Types of Clients:

1. ****Human Clients****: Individuals or businesses that use a service (e.g., a customer using a bank's services).
2. ****Software Clients****: Programs or apps that interact with a server (e.g., a mobile app that connects to a cloud service).

Let me know if you'd like to explore this in a specific context! 😊

[Turn 5 Usage] Input Tokens: 356 | Output Tokens: 216 | Total Tokens: 572

User > /stats

LIVE STATS REPORT

Turn Count:	5
Cumulative Input Tokens:	1063
Cumulative Output Tokens:	495
Cumulative Total Tokens:	1558
Serialized History Length:	2760 characters
Message Stack Count:	11 messages

User > █

7) Why is prior conversation context resent with every turn? How is a system prompt different from a user message? Why do input tokens grow over a conversation? What eventually limits that growth?

LLMs are stateless APIs. They do not maintain an internal persistent memory between distinct HTTP requests. The model would have zero awareness if there was no memory context in each turn of conversation.

The system prompt defines global guidelines, rules, persona, and output constraints. User messages represent dynamic inputs, questions, or commands evaluated within those system-defined rules.

Input Tokens grow because each turn appends both user's new message and the past conversational history stack.

Even though input tokens grow, there are things that can limit that growth: Model Context Window, which are hard token limits set by the architecture, Hardware VRAM limits, which are memory constraints on underlying GPUs, and Cost & Latency.